

High-Level Scalable FTP Analyzer Architecture (100 Gbps)

High-Level Scalable Architecture (100 Gbps)

NIC (100G NIC / Multiple 25G NICs)

-> RSS + rte_flow

RX Queues

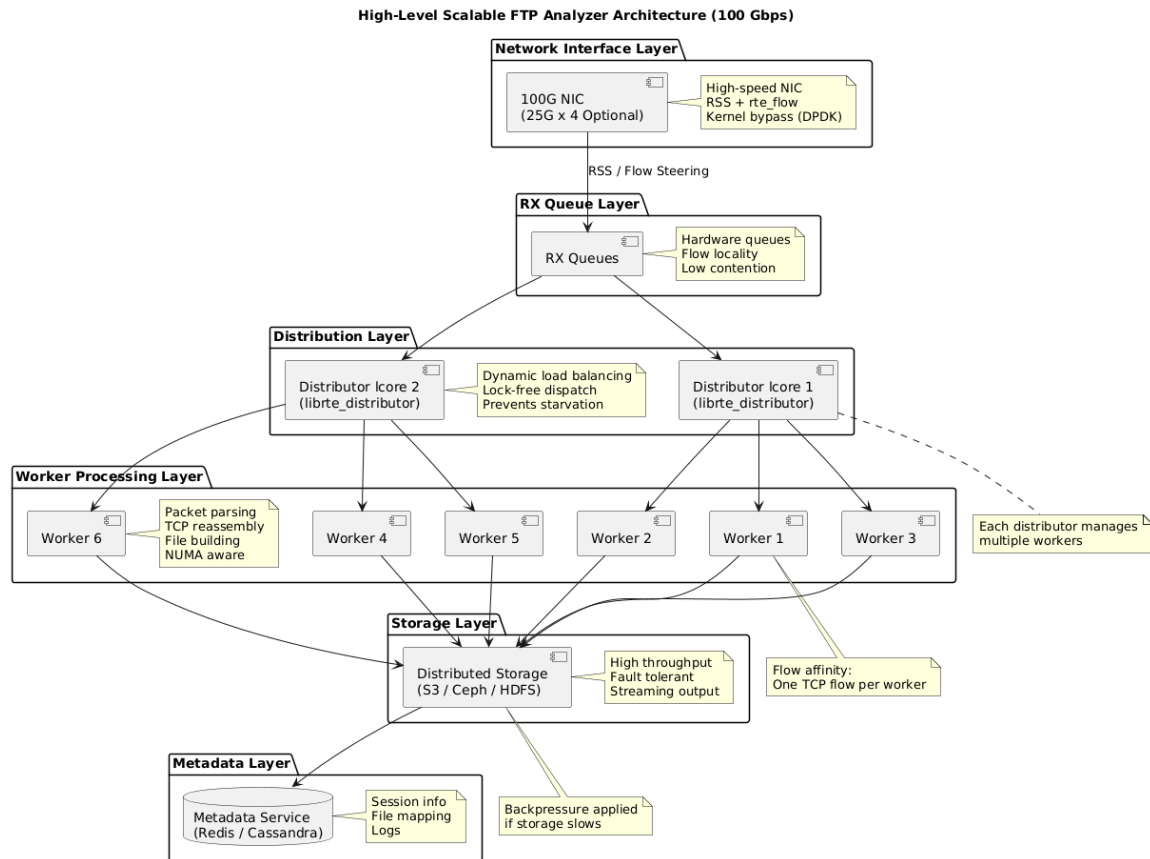
-> Distributor Icores

Worker Icores

-> Distributed Storage

Metadata Service

Architecture Diagram



Design Rationale and Architecture Philosophy

This system is designed to efficiently process and reconstruct FTP traffic at extremely high network speeds, reaching up to 100 Gbps, while maintaining scalability, reliability, and low latency.

To eliminate kernel overhead and achieve line-rate packet capture, the architecture is built using **DPDK-based kernel-bypass networking** on high-speed 100G network interface cards or multiple bonded 25G interfaces. This approach avoids costly *system calls* and *context switches*, allowing packets to be processed directly in **user space**.

Traffic classification and distribution are performed at the hardware level using Receive Side Scaling (RSS) and *rte_flow* rules. These mechanisms steer incoming packets into multiple hardware receive queues based on flow hashing. This enables early parallelization and ensures that traffic is evenly distributed across CPU cores.

Each receive queue is serviced by a dedicated distributor core implemented using *librte_distributor*. Distributor cores dynamically balance the incoming load and dispatch packets to multiple **worker cores** without introducing locking overhead. This design prevents worker starvation and ensures efficient utilisation of processing resources.

Worker cores are responsible for complete flow processing. Each worker independently handles TCP sessions, performs protocol parsing, executes TCP reassembly, manages retransmissions, and reconstructs transferred files. Flow affinity is maintained so that packets belonging to the same session are always processed by the same worker, eliminating synchronization overhead.

To maximize memory efficiency and reduce latency, worker cores are pinned to **NUMA-local CPU** cores and **memory pools**. This minimizes cross-socket memory access, improves cache locality, and provides predictable performance under high traffic loads.

Reconstructed file data is streamed directly to distributed storage systems, providing high-throughput persistence and fault tolerance. Session metadata, transfer state, and operational logs are stored separately in a dedicated metadata service, enabling monitoring, auditing, and recovery.

The architecture supports both horizontal and vertical scalability. Horizontal scaling is achieved by adding more distributor and worker cores, while vertical scaling is enabled by deploying additional high-speed network interfaces or servers. This allows the system to grow incrementally without requiring architectural redesign.

Overall, this design emphasizes parallel processing, zero-copy data handling, flow isolation, and hardware-assisted packet steering. These principles ensure that the system can operate reliably at multi-100 Gbps scale while remaining extensible for future enhancements.

Stage-by-Stage Explanation

Network Interface Layer

- Receive packets at line rate
- Hardware flow steering
- RSS hashing
- `rte_flow` offload

RX Queue Layer

- Buffer packets
- Maintain ordering
- Feed distributors

Distribution Layer

- Dynamic load balancing
- Dispatch to workers
- Avoid idle workers

Processing Layer

- Packet parsing
- TCP reassembly
- File reconstruction
- Session management

Storage Layer

- High throughput writes
- Fault tolerance
- Horizontal scaling

Metadata Layer

- Session tracking
- File mapping
- Monitoring
- Recovery

Key Design Principles

Flow Affinity

One TCP flow per worker

Zero-Copy Processing

Minimize memory copying

NUMA Awareness

Core-memory locality

Horizontal Scalability

Add more nodes

Performance Analysis

- System designed to sustain up to 100 Gbps ingress traffic.
- DPDK kernel bypass reduces syscall and context switch overhead.
- RSS and hardware flow steering enable early parallelization.
- Lock-free distributor minimizes scheduling latency.
- Per-core flow affinity avoids synchronization overhead.
- Zero-copy packet buffers reduce memory bandwidth usage.
- NUMA-aware core pinning improves cache locality.
- Distributed storage supports parallel write throughput.

Performance Optimization Strategies

- Tune RX/TX ring sizes.
- Optimize RSS hash functions.
- Pin workers to dedicated CPU cores.
- Separate memory pools per NUMA node.
- Use hugepages for DPDK buffers.
- Batch packet processing.
- Scale storage nodes independently.

